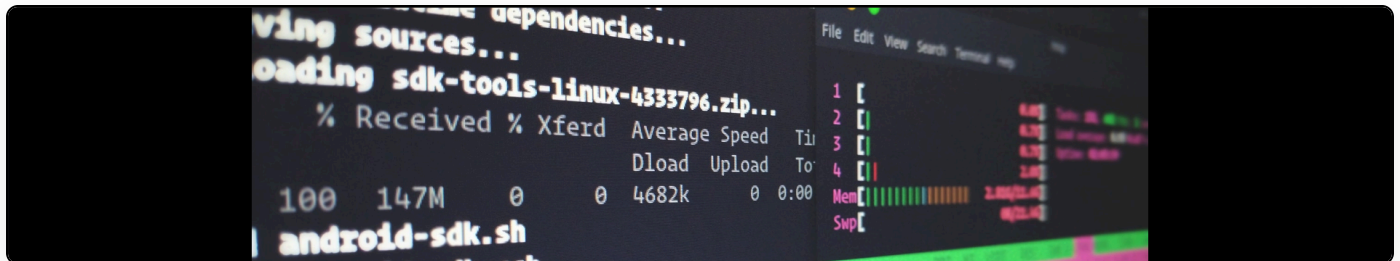


Some lessons in writing Python web scrapers

Cite as: Martin Paul Eve, "Some lessons in writing Python web scrapers", *eve.gd: Martin Paul Eve*, May 18, 2015, <https://doi.org/10.59348/d06dw-jwx30>.



Last weekend I wanted a break from my usual activities, so I decided to write myself some tools to automate a few tasks. One of these is to pull down QIF data from my bank so that I can import it into money management software (I know, I know: I go wild at weekends). I did [a little bit](#) on this a while back but I needed to refresh my memory.

I wanted to share a few observations because my day was largely wasted using the first framework that comes up if you search for “python scraper” wasn’t appropriate to my needs. Namely, I needed a framework that could quickly and dirtily perform a series of actions on a webpage structure. In my use case, this was easier if the framework could in some way use javascript. If you have needs similar to this, my lesson is: **do not use Scrapy; use Selenium.**

Scrapy is a great piece of kit if you want to spider a site and you don’t need javascript. It’s also totally light-weight when compared with Selenium. However, it is painful if your site conducts rigorous checks on form data and all you really want to do is to playback a series of actions masquerading as though you were a web browser. Selenium’s WebDriver is basically a remote control kit for Firefox, Chrome or IE that you drive from the language of your choice. You write bits of code that look like this and it works its magic:

```
aLink = self.driver.find_element_by_id('lstAccLst:0:lkImageRetail1')
aLink.click()

aLink = self.driver.find_element_by_id('pnlgrpStatement:conS2:lkoverlay')
aLink.click()
```

This is, to be frank, totally amazing. Forget having to wrangle with obscure form data and ensuring that you look like a browser. If you're not concerned about performance, then simply *use a browser itself* via Selenium.

I have found that Selenium is not always as robust as Scrapy. If you start multiple instances from the same script, I've had some odd failings. That said, I'm also wrapping my Selenium instance in a virtual displa (using pyvirtualdisplay) so that I don't see the browser, like this:

```
self.display = Display(visible=0, size=(800, 600))
self.display.start()
```